| Life Polaroid |

PM -> curs 5

Preemptive Concurrency -> Parallelism

- MCU -> sunt de obicei SINGLE CORE
- Task-urile în paralel necesita un OS, și pot fi puse pe (pauză) oricând
- Switching-ul între sarcini este costisitor: salvezi contextul (registre, memorie) și îl restaurezi pt alt task
- Task-urile care fac foarte mult I/O -> asta face ca timpul de comutare să dureze mai mult ca TIMPUL DE PROCESARE
  (exp aștepți date de la un senzor) (switching time)

Cooperative Concurrency

- task-urile nu pot fi întrerupte
- componentele hardware funcționează în mod asincron (livrează datele când sunt gata, nu când le ceri)
- task-urile cooperează astfel: un task cedează procesorul când știe că nu mai are ce face (exp așteaptă date)
- nu e nevoie de un OS, totul merge într-un sg flux CONTROLAT MANUAL
- Nu (!) penalizări pentru salvarea/restaurarea stării pt că task-ul știe singur când să "iasă" și "să lase loc" altora.

RP2350 -> este un DUAL CORE MCU, folosim doar un core

! întreruperile hardware (ISR) pot prelua controlul oricând chiar și în COOPERATIVE CONCURRENCY

Asynchronous Executor în Embassy

- executorul gestionează task-urile asincrone
- task-urile pot ceda controlul folosind .await când trebuie să aștepte un eveniment

#[embassy_executor::main]
- marchează funcția principală
- inițializează și pornește scheduler-ul Embassy

Parametrul pool_size -> definește câte copii ale task-ului pot fi active simultan

În task-uri se folosește (.await) ca să eliberezi procesorul cât timp se așteaptă un eveniment (altfel se blochează tot MCU-ul)

Embassy suportă task-uri cu PRIORITĂȚI DIFERITE

- un IRQ (Interrupt Request) de prioritate mare poate lăsa task-uri urgente
- SWI (Software Interrupts) pot lansa task-uri la alte niveluri de priorități
- Task-urile cu priorități mai mici sunt întrerupte dacă vine un task cu prioritate mai mare

Ce este Future în Rust?

Future -> valoare disponibilă în viitor
- echivalentul este PROMISE în java.

Are două stări:
- PENDING -> încă nu e gata
- Ready(T) -> a fost calculat și putem folosi rezultatul T

Un Future are o funcție cheie

fn poll(&mut self) -> Poll<Self::Output>

poll -> executor
-> dacă task-ul este gata, întoarce Poll::Ready(valoare)
-> dacă task-ul nu e gata, întoarce Poll::Pending

Async + Rust
↳ async fn ceva() {}
↳ rust rescrie funcția ta într-un obiect FUTURE
↳ RUST nu rulează singur Future-ul
↳ e nevoie de un executor care să cheme POLL pt FUTURE

CONTEXT și WAKER
↳ poll primește un CONTEXT care conține un WAKER
↳ WAKER-ul e folosit de un task să îi spună executorului „Hei sunt gata acum, și poll-ează-mă din nou"

Embassy + Future
↳ ① Task-urile se înregistrează la executor
↳ ② Când așteaptă un ex. (.await) intră în sleep
↳ un ISR sau un eveniment hardware TREZEȘTE TASK-UL folosind WAKER
↳ executorul rulează task-ul trezit

ACCESUL

① Accesul semultan:
↳ RUST-ul interzice scrierea semultană în același resursă
↳ dacă două task-uri doresc să modifice aceeași resursă trebuie SERIALIZATE

② Accesul EXCLUSIV
↳ se folosește un mecanism de Mutual Exclusion (Mutex)
↳ MUTEXUL se asigură că: -> doar un sg task are voie să scrie la un moment dat
-> celelalte task-uri trb să aștepte

Tipuri de mutex
↳ NOOP MUTEX -> dotă între task-uri cu același executor
↳ CRITICAL SELECTION MUTEX -> dotă între mai multe executoare, ISR, și core-uri
↳ THREAD MODE MUTEX -> dotă între task-uri normale, fără ISR, pe un singur CORE

BLOCKING vs Async Mutex

↳ putem folosi (.await) chiar și când avem lock-ul, MUTEXUL va elibera lock-ul automat cât timp task-ul așteaptă)

↳ Nu putem folosi .await (pt că se blochează complet)

- BLOCKING: SHARED_DATA.lock(|f| {...});
- Async: let mut data = SHARED_DATA.lock().await

Channel-uri

↳ folosite pentru a trimite date între task-uri

① Channel → Multiple Producer Multiple Consumer (MPMC)
② Priority Channel → MPMC, dar cu priorități. Cele urgente trec primele
③ Signal → semnalizează un eveniment, doar ultimul semnal contează, unui sg consumator
④ PubSubChannel → Fiecare consumator primește toate mesajele (broadcast)

CURS 6

ISR (Interrupt Service Routine) = funcție specială APELATĂ automat de când un anumit eveniment hardware are loc
Evenimentele pot fi: → apăsarea unui buton, sosirea unui pachet UART
Când un ISR se întâmplă → procesorul întrerupe ce făcea
↳ sare la ISR-ul care făcea acel eveniment
↳ după ce ISR-ul termină, procesorul reia EXACT unde a rămas

Cum afectează ISR-ul concurența?

① ISR-urile → introduc preemțiune ⇒ un task normal poate fi întrerupt în orice moment dacă un INTERRUPT

② Dacă ISR-ul modifică date comune cu alte task-uri
↳ trebuie protejate cu mecanisme de sincronizare (mutex, secțiuni critice) → altfel apar pb de race condition

DMA → hardware specializat în microcontrolere care face transferuri de date între memorie și periferice fără implicarea procesorului

Plusuri: → scutește MCU de operații tip MEMORY-TO-MEMORY → CPU-ul nu mai trebuie să facă manual (mov, load, store) între adrese
→ MMIO (Memory Mapped I/O) este implicat → transferuri între memorie și periferice (UART, SPI, ADC)
→ când un transfer se termină, DMA ridică un interrupt ca să anunțe CPU-ul că e gata.

! DMA nu știe conținutul cache-ului procesorului
Dacă ai cache activat pe memorie: → zona trb marcată ca "nocache" (dacă MCU permite)
→ fie se face flush manual la cache și după transferul DMA

DMA → eliberează procesorul pt alte calcule, crescând performanța
↳ atenție la sincronizarea MEMORIEI CACHE și FIZICE, pt a nu se transfera date vechi sau incomplete.

UART

↳ metodă comunicare serială între două dispozitive
↳ folosește două fire independente : ① TX → transmitere
② RX → receptie

Firele sunt încrucișate între dispozitive (TX-ul unuia merge la RX-ul celuilalt)

Proprietățile unui dispozitiv UART
① BITS → câți biți sunt în payload (5-9 biți)
② PARITY → se adaugă sau nu un bit de paritate
③ STOP → 1 sau 2 biți de stop
④ BAUD RATE → câți biți sunt trimiși pe secundă (9600, 115200)

Formulă Baud Rate

$$\text{baud Rate} = \frac{f_{clock}}{\text{divider} \times (1 + \text{payload}_{bits} + \text{parity}_{bits} + \text{stop}_{bits})}$$

Tipuri UART
① TTL UART → folosit la comunicații scurte 3.3V sau 5V
② RS232 → pt cabluri lungi între -12V și 12V
③ RS485 → industrial, diferențial (două fire pt diferența de tensiune)

① Receiver (RX) → folosește un Shift Register pt a citi fiecare bit serial
→ poate avea FIFO (coadă pt datele recepționate)
→ generează întreruperi (interrupt) când : → se primește un caracter
→ FIFO-ul este plin sau jumătate plin
② Transmitter (TX) → folosește un Shift Register pt a trimite fiecare bit serial
→ generează întreruperi când : ① se trimite un caracter
② FIFO-ul este gol sau jumătate gol

! ① Duplex complet → se poate transmite/primi simultan
② Nu (!) semnal de ceas transmis între dispozitive → receptorul trebuie să își sincronizeze ceasul intern
③ Pe RP2350 (Raspberry Pi Pico 2 W) → (!) două UART-uri hardware

UART cu Embassy RP

① Synchronous API : `uart.blocking_write(...)`
`uart.blocking_read(...)`

② Asynchronous API (cu DMA și întreruperi) : `uart.write(..).await`

UART = protocolul de bază pt comunicația serială între două dispozitive
Embassy oferă variante sincrone + asincrone pt a lucra eficient cu UART-ul pe microcontrolere

SPI

↳ folosit pentru comunicarea între circuite integrate (microcontroler și senzori)

Avem două tipuri de senzori
① main (master) → controlează comunicarea (MCU-ul)
② sub (slave) → răspunde comenzilor master-ului (de obicei un senzor)

Firele folosite în SPI
① MOSI → Master OUT, Slave IN → date de la master la slave
② MISO → Master IN, SLAVE OUT → date de la slave la master
③ CLK → Clock → ceas generat de master, sincronizează scrierea/citirea
④ CS → Chip Select → activarea individuală a unui slave (nu e standard SPI, dar e esențial în practică)

Cum funcționează o transmisie SPI

① Master setează (CS) pe (LOW) pt după dorit
② Master pune primul bit pe MOSI
③ Slave pune primul bit pe MISO
④ Master pornește CLK (ceasul)
⑤ La rising edge sau falling edge (depinde de mod), datele sunt citite/scrise

CPOL și CPHA definesc când anume sunt copiate și transmise datele în raport cu semnalul de ceas (CLK)

MODE	CPOL	CPHA	SEMNIFICAȚIE
0	0	0	Rising Edge, la înc ciclului
1	0	1	Rising Edge, la mijlocul ciclului
2	1	0	Falling edge, la înc ciclului
3	1	1	Falling edge, la mijlocul ciclului

- Daisy Chaining -> = conectează mai multe slave-uri în lanț: ① Masterul trimite date către primul slave
② Fiecare slave trimite mai departe datele către urm
↳ = utilizează mai ales LED-uri adresabile sau unele ADC-uri

- SPI → protocol serial, rapid, pentru conectarea mai multor disp periferice la un controler

- Embassy oferă atât metode sincrone și asincrone (cu suport DMA) pt a face transmisii eficiente).

! Imp SPI
① Transmisie → duplex complet
② Clock → sursă unică de ceas de la master
③ Nr. dispozitive → 1 master + mai mulți slave
④ Viteză → nu (!) limită teoretică, limitată practic de hardware

SENZORI

- Tipuri de senzori:
  ① analog senzor: doar traductor (un senzor de temperatură simplu), necesită un ADC pentru conversie digital
  ② digital senzor: traductor + ADC + procesor MCU integrat, necesită ieșire digitală pe SPI / I2C

ANALOG = semnal de tensiune 2gomotos → NECESITĂ PROCESARE
DIGITAL = semnal digital gata de folosit → prin magistrala SPI sau I2C

Exp senzor digital: BMP280
↳ senzor digital fabricat de BOSCH
Are: → BUS SPI sau I2C
→ Registru de control pt config
→ Registru de date pt citirea presiunii

CITIRE DATE dintr-un senzor digital prin SPI
① pui linia CHIP SELECT (CS) pe LOW (activezi senzorul)
② trimiti adresa registrului dorit + bitul de citire (1<<7)
③ primești răsp prin SPI
④ Ridici CS înapoi pe HIGH

! ① Senzorii analogici = necesită ADC și procesare 2gomot
② Senzori digitali = comunicare magistrală SPI sau I2C
③ Citire SPI = trimitem adresă cu bit citire setat
④ Scriere SPI = trimitem adresă cu bit de scriere
⑤ Embossy = permite atât operații sincrone cât și asincrone cu SPI

I2C

↳ folosit pt comunicarea între circuite integrate

Are două roluri

① CONTROLLER (master) → inițiază comunicarea (exp: microcontrolerul)

② TARGET (slave) → răspunde comenzilor master-ului (exp senzor)

Firele și adresele

① SDA : Serial Data Line → pt date bidirecționale

② SCL : Serial Clock Line → semnal de ceas generat de controller

Fiecare dispozitiv target are o adresă unică (7 sau 10 biți)

Liniile sunt PULL-UP (HIGH) și sunt trase în jos (LOW) când se scrie

Cum funcționează o transmisie I2C → 7 bit address

① controllerul trimite semnalul START

② controllerul trimite adresa dispozitivului TARGET + bit de citire/scriere (R/W)

③ Target-ul răspunde cu ACK sau NACK

④ Controllerul sau targetul transmite datele

⑤ Controllerul transmite semnalul STOP

10-bit address

↳ similar dar se trimit două coduri pt adresa completă de 10 biți

! Transmisie = half-duplex (cum sg sens la un moment dat)

Ceas = sincronizat de controller

Fire = SDA/SCL (aceleași fir de culine/servere) și un fir pt ceas

Dispozitive = un controller, mai multe target-uri (un receiver, un transmitter)

I2C → protocol serial, a fire master-slave

SDA/SCL → linie date și linie ceas

Adrese → 7 sau 10 biți

Moduri → viteză până la 5 Mbit/s (Ultra-fast)

Embossy → suportă atât operații sincron cât și asincron pt I2C

Folosit la: → senzori

→ display-uri mici

→ RP2350 are două dispozitive I2C

USB (universal Serial BUS)

- folosit pentru comunicatie intre un host (calculator) și mai multe device-uri (mouse, tastatură, stick USB)
- un dispozitiv primește o adresa 7-bit la conectare
- putem conecta până la 127 de dispozitive la un host, folosind HUBS.
- structura este IERARHICĂ: HOST -> HUB -> DEVICE-URI

Arhitectura de comunicare

1. HOST -> inițiază comunicarea
2. DEVICE -> răspunde la cererile hostului
3. ENDPOINTS -> buffere pt date IN and OUT
4. BUS -> 2 fire de date (DP/DM) folosind semnal diferential
5. CLOCK -> 48 MHz intern (MCU -> sincronizeaza local)

Tipuri pachete USB

1. TOKEN -> inițiază transferul (OUT, IN, SETUP)
2. DATA -> transportă date reale (DATA0, DATA 1)
3. HANDSHAKE -> confirmă transferul (ACK, NACK, STALL)

Moduri de transfer

1. control = configurare dispozitiv, folosit pt INITIALIZARE, SETARE, DESCRIPTOR
2. ISOCHRONOUS = viteză mare, fără garanția integrității, folosit pt audio/video streaming
3. BULK = fiabilitate, viteză mai mică, folosit pt transfer de fișiere mari
4. Interrupt = latență garantată, folosit pt mouse, tastaturi

Descriptori USB

- device descriptor -> inf generale despre dispozitiv
- configuration descriptor -> configurații posibile
- interface descriptor -> funcții / servicii oferite de dispozitiv
- endpoint descriptor -> cum sunt transferate datele

Viteze USB

- low speed -> 1.5 Mbit/s
- full speed -> 12 Mbit/s
- high speed -> 480 Mbit/s

Embassy + USB API pe RP2350

- config dispozitivului: `embassy_usb::Builder`
- crearea endpointurilor: `bulk_in`, `bulk_out`

API-uri pt host (PC)
- folosind crate-ul `NUSB`

① USB Transfer = TOKEN → Data → Handshake
② dispozitive = host utilizează, device răspunde
③ descriptori - DEVICE, CONFIGURATION, INTERFACE, ENDPOINT
④ embassy USB = putem configura device-uri USB embedded în RUST
⑤ Moduri de transfer = CONTROL, BULK, INTERRUPT, ISOCHRONOUS

Senzori I2C

- Tipuri de senzori
  - ① analog → doar traductor analogic (exp variază tensiunea) → necesită ADC și filtrare zgomot
  - ② digital → traductor + ADC + microcontroler intern. Livrează date direct digital, prin magistrală (I2C/SPI)

Exp senzor digital: BMP280 (senzor presiune de la Bosch)
- ↳ comunică prin I2C sau SPI
- ↳ INTERN: traductor presiune → ADC → MCU intern → output digital

Harta regiștrilor BMP280
- ↳ regiștri importanți: ① ctrl_meas (0xF4) - configurezi modul de măsurare
  ② press_lsb (0xF8) - citești valoarea presiunii

write_read = citim date dintr-un registru
write = scriem date într-un registru

LAB 2 -> GPIO

↳ **General-Purpose Input/Output** -> este o parte esențială a sistemelor embedded -> este esențial în legătura dintre MICROCONTROLERE și MICROPROCESOARE cu lumea exterioară

Pinii unui microcontroller sau a unui microprocesor pot fi configurați ca intrări sau ieșiri
(Permit comunicarea cu componentele externe (LED-uri, butoane, senzori, etc)

Protocoalele standard de comunicare sunt SPI, I2C, PCM, PWM, iar comunicarea serială e facută cu ajutorul pinilor GPIO.
Sunt două tipuri de pini: digitali și analog.

Configurarea pinilor GPIO

↳ pot să fie folosiți ca: LED-uri, motoare, etc

Un pin poate fi configurat fie ca "INTRARE", fie ca "IEȘIRE" -> dar nu ambele în același timp

Cod aprindere led:

anunțăm programul la LED-ul e conectat la PINUL numit LED

① `let mut led = pins.led.into_push_pull_output();`

transf pinul în ieșire (output) ai să putem aprinde sau stinge ledul

`led.set_high();`
`led.set_low();`
↳ trimitem 0 (oprim curentul) = oprim LED-ul
↳ trimitem 1 (curent) = aprindem LED-ul

la pinul is e conectat butonul

->configurăm ca intrare și activăm „pull up resistor" (fără apăsare = pinul e automat 1/High)

② `let button = pins.gpio1s.into_pull_up_input();`

```
if button.is_low().unwrap(){
    // butonul a fost apăsat
}
```

↳ verif dacă semnalul e jos

-> dacă butonul e apăsat, circuitul se închide => semnalul devine 0 => codul din {} se execută

(!) fenomenul de DEBOUNCE la butoane -> când apăsăm un buton din cauza vibrațiilor mecanice, semnalul nu trece frumos de la 1 la 0
Altfel se produc erori
Soluție: așteptăm câteva milisecunde

`Timer::after_millis(20).await // asteapta 20 ms`

Folosirea ASYNC/AWAIT pentru multitasking
Fără să scriem loop, putem scrie mai multe task-uri care merg în paralel
Cu `#[embassy_executor::task]` și await -> pt a aștepta fără să blocăm procesul

```
#[embassy_executor::main]
async fn main(_spawner: Spawner){
    let p = embassy_rp::init(Default::default()); -> inițializează placuța
    let mut led = p.PIN_25.into_push_pull_output();
    loop {
        led.set_high(); // aprind ledul
        Time::after_millis(500).await, // astept 5s
        led.set_low();          // sting ledul
        Time::after_millis(500).await;    // astept 5s
    }
}
```

Spawn → lansează două sarcini care merg în paralel

Cum funcționează GPIO în RP2040/RP2350?

① Pads → controlează pinul fizic: curent maxim, rezistențe de pull-up/down
② IO Bank0 → face legătura între perifericele interne și pini

GPIO poate fi folosit de:
① SIO (funcție de GPIO) → controlează pinii interni, și e cel mai des folosit de programatori
② SPI_RX
③ I2C0_SDA
④ UART0_TX

Multiplexare - fiecare PIN poate avea mai multe funcții - IO Bank0 decide ce rol are pinul în acel moment

① embassy-executor - execută tasksuri async
② embassy-time - oferă delay-uri și timere
③ embassy-usb, embassy-net → comunicare USB/Network
④ defmt → logging eficient pt embedded
⑤ cortex-m-rt, cortex-m → runtime pt ARM, cortex-M
⑥ rp_pac → acces direct la regiștrii (PAC)

LAB 3

① Semnale: ① digitale: 0 sau 1 (două nivele)
② analogice: valori continue (sunet, lumină)

② PWM → simulează un semnal analogic folosind doar 0 și 1
→ schimbăm „cât de mult timp" semnalul stă sus (high)

③ ADC → transformă un semnal analogic (tensiune) în date digitale

④ Ceas → circuit care oscilează constant între HIGH și LOW
→ cel mai precis ceas: oscilator cu cristal (XOSC)

⑤ Contoare → țin evidența nr de impulsuri pe ceas
→ când ajung la maxim, revin la 0 (overflow)
→ 3 regiștri importanți: value → valoare curentă
direction → UP sau DOWN
reset → când se resetează

Un counter de 8 biți numără de la 0 la 255, apoi revine la 0

① SYST_CVR → valoarea actuală a timerului
② SYST_RVR → valoarea la care se resetează
③ SYST_CSR_SET → control (pornire/oprire, întreruperi)

⚠ RP2350 → are două periferice timer (fiecare de câte 4 la RP2040)
→ timerele sunt MONOTONICE (nu dau overflow)

PWM = Pulse Width Modulation

let pwm = &mut p.PWM_CH6
<-> PWM pe canalul 6

set_divider(1) = nu scadem frecvent - folosim frecventa completa (125 MHz)
set_top(100) = PWM numara de la 0 la 100, deci avem o rezolutie de 1%

set_compare(...) : cat din TOP e semnal HIGH

led.set_function(PinFunction::PWM) = ii spunem pinului sa foloseasca functia PWM, nu GPIO normal

ADC -> Analog to Digital Converter = se citeste un semnal analogic si se transforma in val digitala.

let adc = &mut p.ADC
adc.enable = pornim perifericul ADC
let mut pin = p.PIN_28 => pinul conectat la potentiometru

pin.set_function(PinFunction::Null) -> pinul nu mai e GPIO digital -> devine analogic

adc.read(&mut pin) -> citim o valoare de la 0 la 4095 (pe 12 biti)

PWM -> controleaza semnale digitale pt a simula valori analogice (exp: intensitate LED)
ADC -> citeste tensiuni analogice si le transforma in numere (ex unui potentiometru)

LAB 4

Ce e un async fn? -> o functie care „pune pauza" si spune „Astept ceva, dar intre timp fa altceva"
Task-ul asincron e pornit de: spawner.spawn(...)

button.wait_for_falling_edge() -> butonul e apasat

join! -> asteapta tot

```
join!(
    task_led(),
    task_buton(),
    task_uart()
);
```

① async fn -> functie care poate fi intrerupta
② .await -> asteapta fara sa blocheze tot programul
③ Future -> o promisiune ca se va intampla ceva in viitor
④ Executor -> manager care coordoneaza ce task ruleaza cand
⑤ spawn() -> porneste un task nou
⑥ select! -> alege primul FUTURE! care termina
⑦ join! -> asteapta toate Future-urile

LAB 5

SPI → protocol de comunicare sincron, rapid, între
① un dispozitiv principal (main / master)
② unul sau mai multe sub-dispozitive (sub / slave)

Linii utilizate

① CLK → clock-generat de master
② MOSI → Master Out Slave In → date de la master la slave
③ MISO → date de la slave la master
④ CS → Chip Select → activează un slave, fiecare slave are propriul CS

Protocolul este FULL-DUPLEX → se transmit datele în ambele direcții simultan

Slot LOW → se activează

Exp transmisie SPI
① Masterul setează CS pe LOW → activează slave-ul
② Master scrie un bit pe MOSI, iar slave-ul scrie un bit pe MISO
③ Master pornește ceasul CLK
④ La fiecare "rising edge" sau "falling edge" → se face citirea / scrierea semnalului
⑤ Se repetă până la final

DMA → Direct Memory Acces
① DMA → mută date fără implicarea procesorului
② Poate transmite / cati date SPI fără să blocheze firul principal
③ Când termină trimite un INTERRUPT

LAB 6

I2C → Inter-Integrated Circuit
↳ protocol bidirectional și folosește doar două fire
① SDA (date)
② SCL (ceas)

Un dispozitiv CONTROLLER (master) și unul sau mai mulți target (slave)
! Masterul inițiază comunicarea

① SDA (Serial data) → transmite bit cu bit datele, fie de la master la slave fie invers
② SCL (serial clock) → masterul generează semnalul de ceas
Fiecare slave are o adresa de 7 biți (sau 10 biți)

Pull-up → liniile sunt trase în sus (High) prin rezistențe, dispozitivele „trag" în jos (LOW) pentru a transmite

① Half-duplex -> nu trimiți și primești simultan (folosești același fir SDA)
② Cine semnează -> master și slave folosesc același ceas
③ Pe RP2350 sunt două periferice I2C (I2C0, I2C1)

```
let tx_buf = [0x90u8]
i2c.write(0x5E, &tx_buf).unwrap(); // .unwrap() -> pentru a opri la eroare

// altfel e ceilalți: let mut rx_buf: [0u8; 7]
```

① new_blocking(_) -> creează un driver I2C care blochează firul de execuție până se termină transferul
② write(addr, &tx_buff) -> trimite tx_buf către dispozitivul cu adresa addr
③ read(addr, &mut rx_buf) -> citește în rx_buf

Curs 3+4 → GPIO

Operații bitwise fundamentale

// setarea bitului pe o poziție specificată

```
fn set_bit (register: usize, bit: u8) -> usize {
    register | (1<<bit)
}
```

// setarea mai multor biți odata

```
fn set_bits (register: usize, bits: usize) -> usize{
    register|bits
}
```

STERGEREA BIȚILOR

// Stergerea bitului pe poziția specificată

```
fn clear_bit ( register: usize, bit:u8) -> usize {
    register & !(1<<bit)
}
```

// stergerea mai multor biți

```
fn clear_bits ( register: usize, bits: usize) -> usize {
    register &!bits
}
```

// inversarea bițiilor pe poziția specificată

```
fn flip_bit (register: usize, bit: u8) -> usize {
    register ^ (1<<bit)
}
```

// extragerea valorilor din registru

```
const MASK: u32 = 0b0000_0000_0000_0000_0000_1111_1111_1111;
let extracted_bits = (large_id >>20) & MASK;
```

Memory Mapped I/O (MMIO)

Conceptul de bază : → perifericele sunt accesate ca zone de memorie
→ fiecare registru are o adresă specifică
→ se folosesc pointeri pentru accesarea registrelor

GPIO pe RP2040

↳ Arhitectura GPIO
GPIO-urile pe RP2040 folosesc 3 PERIFERICE

① SIO (Single Cycle IO) - Controlează input / output

② IO BANK → Multiplexează funcțiile pinilor

③ PAD - Configurează parametrii electrici

Concepte importante despre semnale

- NOISE MARGIN → Spațiul de siguranță pentru semnale
- Pull-up / Pull-down → Rezistori pentru stări definite
- Level shifting → Pentru compatibilitatea între tensiuni diferite

Exercitii -> Operatii Bitwise

registru: 0b1000 | -> set_bit
bit = 2

Pas 1: calculez 1<<bit

1<<2 ?

Scriu 1 in binar = 0001
Shift left cu 2 pozitii: 0100

Deci 1<<2 = 0b0100

Pas 2: Calculez registru | (1<<bit)

registru = 0b1000
1<<bit = 0b0100

Operatia OR = 0b1100

Rezultat = 0b1100

CLEAR BIT

registru = 0b1100, bit = 2

Pas 1: Calculez 1<<bit

1 << 2 = 0b0100 (din ex anterior)

Pas 2: Calculez ! (1<<bit)

0b0100 NOT = 0b1011

Pas 3: Calculez registru & ! (1<<bit)

registru = 0b1100
!(1<<bit) = 0b1011

AND = 0b1000

Rezultat: 0b1000

Extragerea de biți

```
┌──────────────────────────────────────────────────┐
│ EXERCIȚIU: Extragere porțiune din ID             │
│ ID = 0b1100_1010_1111_1100_0000_1111_0110_1101   │
│ Vreau să extrag biții 31:20                      │
└──────────────────────────────────────────────────┘

Pas 1: Desenez ID-ul cu poziții
Pozițiile: 31 30 29 28 27 26 25 24 23 22 21 20 19 18...
ID:        1  1  0  0  1  0  1  0  1  1  1  1  1  1...

Zona pe care o vreau (31:20):
 1  1  0  0  1  0  1  0  1  1  1  1

Pas 2: Shift right cu 20 poziții
ID >> 20 = ?

Original (32 biți):
1100_1010_1111_1100_0000_1111_0110_1101

După >> 20 (umplu cu 0 la stânga):
0000_0000_0000_0000_0000_1100_1010_1111

Pas 3: Aplic masca
MASK = 0b0000_0000_0000_0000_0000_1111_1111_1111

(ID >> 20):  0000_0000_0000_0000_0000_1100_1010_1111
MASK:        0000_0000_0000_0000_0000_1111_1111_1111
             ____________________________________
Rezultat:    0000_0000_0000_0000_0000_1100_1010_1111

REZULTAT FINAL: 0b1100_1010_1111 (biții 31:20)
```